

Polyglot Persistence Design Document

Healthcare Patient Management Platform - GP3

Team 6: Louis Tafah, Pushkar Vishwas, Adyasha Mishra, Etsubdink Workalemahu Yergashewa

Part 1: Data Partitioning Analysis

Summary Table: Data Remaining in PostgreSQL (Unchanged from GP2)

Data Type	Database	Rationale
Patient Demographics	PostgreSQL	Structured, ACID required, access-controlled PHI
Provider Information	PostgreSQL	Structured, referential integrity with departments
Hospital & Department	PostgreSQL	Hierarchical structure, strong relationships
Appointments	PostgreSQL	Complex relationships (patient, provider), scheduling constraints
Prescriptions	PostgreSQL	Strong consistency required, legal/audit requirements
Medications	PostgreSQL	Structured catalog, referenced by prescriptions
Insurance Plans	PostgreSQL	Financial accuracy, ACID transactions
Patient Insurance	PostgreSQL	Policy data with financial constraints
Billing Records	PostgreSQL	Financial accuracy, balance calculations
Claims	PostgreSQL	ACID required, audit trail
Lab Tests & Results	PostgreSQL	Structured, normal ranges, referential integrity
Admissions	PostgreSQL	Complex relationships, date constraints
Emergency Contacts	PostgreSQL	Structured, linked to patient
Allergies	PostgreSQL	Structured, severity levels, linked to patient

Decision Templates for MongoDB Data Types

Data Type: Clinical Notes **Analysis:** 1. **Structure:** Variable - progress notes, consultations, discharge summaries, and procedure notes have entirely different field sets. A progress note includes vitals and physical exam sections; a consultation has requesting provider and recommendations; a discharge summary has hospital course and discharge medications. 2. **Consistency:** Eventual consistency acceptable - notes are append-mostly (rarely edited after signing), and clinical workflows do not require immediate cross-system consistency. 3. **Relationships:** Minimal - only references patient_id and provider_id from PostgreSQL. The note content is self-contained. 4. **Query Pattern:** Document retrieval by patient timeline, full-text search across narratives (symptoms, diagnoses, plans), filtering by note type and ICD-10 codes. 5. **Volume:** Moderate - typically 10-50 notes per patient per year, with text-heavy content unsuitable for row storage. 6. **Evolution:** High - new note templates (telehealth notes, nursing assessments) added as clinical practice evolves.

Decision: MongoDB

Justification: Clinical notes have inherently variable structure depending on note type. Progress notes require vitals and physical exam sections, while consultations need referral information. MongoDB's flexible schema accommodates these differences without NULL-heavy tables. Text search capabilities enable searching clinical narratives for symptoms and diagnoses. The append-mostly nature of signed notes aligns with eventual consistency.

Data Type: Medical Imaging Metadata Analysis: 1. **Structure:** Variable - DICOM headers differ significantly by imaging modality. CT scans have slice thickness and reconstruction kernels; MRIs have sequence type and field strength; mammograms have BIRADS ratings; X-rays have view positions. 2. **Consistency:** Eventual consistency acceptable - imaging metadata is write-once after study completion. 3. **Relationships:** Minimal - references patient_id and ordering provider_id. Radiologist reports are embedded within the same document. 4. **Query Pattern:** Patient imaging timeline lookup, filtering by modality and body part, critical finding alerts, text search on radiologist impressions. 5. **Volume:** Moderate - imaging studies generate rich metadata but are bounded per patient (typically 2-10 studies per year). 6. **Evolution:** Moderate - new imaging modalities and DICOM fields emerge as technology advances.

Decision: MongoDB

Justification: DICOM metadata varies dramatically by modality. A rigid SQL schema would require either many Nullable columns or a complex EAV pattern. MongoDB's document model naturally accommodates modality-specific fields (slice_thickness for CT, sequence for MRI, birads for mammography) without schema changes. Embedding the radiologist report within the imaging document keeps related data together for efficient retrieval.

Data Type: Care Plans Analysis: 1. **Structure:** Variable - care plans contain nested arrays of goals and interventions that vary by diagnosis and patient complexity. A diabetes care plan has different goals than a heart failure care plan. 2. **Consistency:** Eventual consistency acceptable - care plans are reviewed periodically (weekly/monthly) and updates do not require immediate propagation. 3. **Relationships:** Minimal - references patient_id and provider_id. Goals and interventions are embedded within the care plan. 4. **Query Pattern:** Active care plan lookup by patient, goal progress tracking, filtering by diagnosis (ICD-10), provider workload analysis. 5. **Volume:** Low to moderate - typically 1-5 active care plans per patient with complex chronic conditions. 6. **Evolution:** High - care protocols evolve with clinical guidelines and quality measures.

Decision: MongoDB

Justification: Care plans are hierarchical documents with variable numbers of goals and interventions. Each goal has its own status and progress tracking. Embedding these arrays within the care plan document maintains data locality and enables atomic updates to individual goals. MongoDB's aggregation framework supports goal progress analytics without complex JOINS.

Data Type: Patient Surveys Analysis: 1. **Structure:** Variable - different survey instruments have different questions and scoring algorithms. PHQ-9 has 9 depression questions; GAD-7 has 7 anxiety questions; pain scales and satisfaction surveys have entirely different structures. 2. **Consistency:** Eventual consistency acceptable - surveys are completed at point of care and analyzed in batch for trends. 3. **Relationships:** Minimal - references patient_id. Survey responses are self-contained. 4. **Query Pattern:** Patient survey history, score trending over time, risk stratification by severity, response-level analysis for clinical screening. 5. **Volume:** Moderate - patients may complete surveys at each visit (4-12 times per year). 6. **Evolution:** High - new validated instruments added frequently, existing instruments updated.

Decision: MongoDB

Justification: Survey instruments are inherently document-oriented with variable question sets. A PHQ-9 response structure differs from a GAD-7 or pain assessment. Embedding responses within the survey document enables complete retrieval of a survey with all its answers. MongoDB's aggregation pipelines support score trending and severity analysis without complex SQL.

Decision Templates for Neo4j Data Types

Data Type: Medications (Knowledge Graph) Analysis: 1. **Structure:** Semi-structured - medications have properties (name, generic, form, schedule) but exist primarily for relationships. 2. **Consistency:** Strong consistency not required - drug knowledge is reference data updated infrequently. 3. **Relationships:** Many and critical - medications belong to drug classes, interact with other medications, are contraindicated for allergies and diseases. These relationships are the primary use case. 4. **Query Pattern:** Multi-hop traversals for interaction checking, class-level interaction analysis, finding treatment alternatives. 5. **Volume:** Relatively small - hundreds to thousands of medications in a knowledge graph. 6. **Evolution:** Moderate - new drugs added, interactions updated based on post-market surveillance.

Decision: Neo4j

Justification: Medications are defined by their relationships - drug classes, interactions, contraindications. A relational model would require expensive JOIN operations across multiple tables to check interactions. Neo4j's graph traversals efficiently find all medications that interact with a patient's current medication list in a single query, enabling real-time clinical decision support.

Data Type: Drug Interactions Analysis: 1. **Structure:** Relationship-centric - interactions are bidirectional edges between medication nodes with severity and description properties. 2. **Consistency:** Not applicable - interactions are reference knowledge, not transactional data. 3. **Relationships:** This IS the relationship - drug interactions are inherently edges in a graph. 4. **Query Pattern:** "Given medications A, B, C, find all pairwise interactions and their severities" - a classic graph pattern match. 5. **Volume:** Moderate - thousands of known drug-drug interactions. 6. **Evolution:** Ongoing - new interactions discovered through clinical research and FDA alerts.

Decision: Neo4j

Justification: Drug interactions ARE relationships. Modeling them as graph edges with severity properties enables efficient bidirectional traversal and pairwise interaction checking. The prescription safety check queries all current medications against a proposed new medication in a single Cypher query, returning interaction alerts for clinical decision support.

Data Type: Diseases Analysis: 1. **Structure:** Semi-structured - diseases have properties (name, ICD-10, category, chronic/acute) and serve as hubs for clinical relationships. 2. **Consistency:** Not required - disease knowledge is reference data. 3. **Relationships:** Extensive - diseases present with symptoms, are treated by medications, and some medications are contraindicated in certain diseases. 4. **Query Pattern:** Clinical pathway traversals (disease -> treatment options), symptom-based differential diagnosis, contraindication checking. 5. **Volume:** Small - hundreds of commonly encountered diseases. 6. **Evolution:** Stable - ICD-10 codes and disease definitions change slowly.

Decision: Neo4j

Justification: Diseases are relationship-rich entities. A disease node connects to symptom nodes (for diagnostic support), medication nodes (for treatment pathways), and back to medications via contraindica-

tion edges. These multi-hop traversals power clinical decision support features that would require complex recursive CTEs in SQL.

Data Type: Symptoms **Analysis:** 1. **Structure:** Semi-structured - symptoms have properties (name, body system, severity range) and connect to diseases. 2. **Consistency:** Not required - symptom knowledge is reference data. 3. **Relationships:** PRESENTS_WITH edges connect diseases to their symptoms with frequency properties (common, uncommon, rare). 4. **Query Pattern:** “Given symptoms X, Y, Z, what diseases present with this combination?” - a graph intersection query. 5. **Volume:** Small - hundreds of clinical symptoms. 6. **Evolution:** Stable - clinical symptom vocabulary is well-established.

Decision: Neo4j

Justification: Symptoms exist to support disease differential diagnosis through relationship traversal. Finding diseases that match a symptom pattern requires intersecting graph paths, which Neo4j handles efficiently. This enables future clinical decision support for diagnostic assistance.

Data Type: Provider Referral Network **Analysis:** 1. **Structure:** Relationship-centric - providers connect via REFERS_TO edges with referral counts and common reasons. 2. **Consistency:** Eventual - referral patterns are analyzed for network optimization, not real-time decisions. 3. **Relationships:** Primary use case - understanding referral patterns between specialties and providers. 4. **Query Pattern:** “Who are the top referral destinations for Family Medicine?”, network analysis for care coordination. 5. **Volume:** Small - dozens to hundreds of providers in a health system. 6. **Evolution:** Dynamic - referral patterns change as providers join/leave.

Decision: Neo4j

Justification: Provider referral networks are inherently graph structures. Analyzing referral patterns, finding referral paths between specialties, and identifying network bottlenecks are natural graph queries. The REFERS_TO relationships capture referral volume and common reasons, enabling healthcare network optimization.

Data Type: Allergens and Contraindications **Analysis:** 1. **Structure:** Semi-structured - allergens have name and category; contraindications are edges to medications. 2. **Consistency:** Not required - allergy knowledge is reference data. 3. **Relationships:** CONTRAINDICATED_FOR edges connect medications to allergens, ALLERGIC_TO edges connect patients to allergens. 4. **Query Pattern:** “Given patient allergies, which medications are unsafe?” - a two-hop traversal. 5. **Volume:** Small - common drug and food allergens. 6. **Evolution:** Stable - allergen categories well-established.

Decision: Neo4j

Justification: Allergy-based medication safety checking requires traversing from patient to allergens to contraindicated medications. Neo4j efficiently executes this pattern: MATCH (p:Patient)-[:ALLERGIC_TO]->(a:Allergen)<-[:CONTRAINDICATED_FOR]-(m:Medication). This powers the prescription safety check preventing allergic reactions.

Part 2: MongoDB Schema Design

Collection 1: clinical_notes

Purpose: Store clinical documentation with flexible structures across progress notes, consultations, discharge summaries, and procedure notes.

Document Structure: - `_id`: ObjectId, auto-generated by MongoDB - `patient_id`: Integer, references PostgreSQL `patient.patient_id` - `provider_id`: Integer, references PostgreSQL `provider.provider_id` - `encounter_date`: Date of the clinical encounter - `note_type`: String enum - `progress_note`, `consultation`, `discharge_summary`, `procedure_note` - `icd10_codes`: Array of diagnosis codes used for clinical filtering - `signed`: Boolean indicating whether the note is finalized - `signed_date`: Date when the note was finalized, or null for unsigned notes - Progress-note fields: `chief_complaint`, `review_of_systems`, `physical_exam`, `assessment`, `plan` - Consultation fields: `requesting_provider_id`, `reason_for_consultation`, `findings`, `recommendations`, `urgency` - Discharge-summary fields: `admission_date`, `discharge_date`, `discharge_diagnosis`, `hospital_course`, `discharge_medications`, `follow_up` - Procedure-note fields: `procedure_name`, `indication`, `anesthesia`, `findings`, `complications`

Sample Document (Progress Note):

```
{
  "_id": ObjectId("..."),
  "patient_id": 1,
  "provider_id": 4,
  "encounter_date": ISODate("2026-03-15"),
  "note_type": "progress_note",
  "chief_complaint": "Follow-up for hypertension management",
  "review_of_systems": {
    "cardiovascular": "No chest pain, no palpitations",
    "respiratory": "No shortness of breath"
  },
  "physical_exam": {
    "vitals": {
      "bp_systolic": 138,
      "bp_diastolic": 85,
      "heart_rate": 72,
      "temperature": 98.6
    },
    "findings": "Heart regular rate and rhythm, lungs clear"
  },
  "assessment": "Hypertension, improving on current regimen",
  "plan": "Continue lisinopril 10mg daily, recheck in 3 months",
  "icd10_codes": ["I10"],
  "signed": true,
  "signed_date": ISODate("2026-03-15T16:30:00Z")
}
```

Embedding Rationale: Review of systems, physical exam, and vitals are embedded because they are bounded (fixed structure within a note type) and always viewed together with the parent note. Patient and provider data are referenced by ID because they already exist as strongly consistent PostgreSQL records requiring ACID guarantees.

Expected Volume: ~240 documents seeded; production would see 10-50 notes per patient per year.

Collection 2: `medical_images_metadata`

Purpose: Store imaging study metadata and radiologist reports while allowing modality-specific DICOM fields.

Document Structure: - `_id`: ObjectId - `patient_id`: Integer, references PostgreSQL `patient.patient_id` - `provider_id`: Integer, ordering or associated provider - `study_date`: Date of imaging study - `modality`: String enum - CT, MRI, XRAY, US, MAMMO - `body_part`: Study body region - `study_description`:

Human-readable study description - dicom_metadata: Embedded object for modality-specific fields such as study_uid, series_count, image_count, slice_thickness, sequence, views, birads - radiologist_report: Embedded object containing provider_id, findings, impression, critical_finding, and signed_date

Sample Document (CT Scan):

```
{
  "_id": ObjectId("..."),
  "patient_id": 5,
  "provider_id": 8,
  "study_date": ISODate("2026-02-10"),
  "modality": "CT",
  "body_part": "Chest",
  "study_description": "CT Chest with contrast",
  "dicom_metadata": {
    "study_uid": "1.2.840.113619.2.55.3.123456789",
    "series_count": 3,
    "image_count": 245,
    "slice_thickness": 1.25,
    "contrast_used": true,
    "reconstruction_kernel": "STANDARD"
  },
  "radiologist_report": {
    "provider_id": 8,
    "findings": "No pulmonary embolism. Heart size normal. No pleural effusion.",
    "impression": "Normal CT chest with contrast",
    "critical_finding": false,
    "signed_date": ISODate("2026-02-10T14:30:00Z")
  }
}
```

Sample Document (Mammogram):

```
{
  "_id": ObjectId("..."),
  "patient_id": 12,
  "provider_id": 8,
  "study_date": ISODate("2026-01-20"),
  "modality": "MAMMO",
  "body_part": "Bilateral Breasts",
  "study_description": "Screening Mammogram",
  "dicom_metadata": {
    "study_uid": "1.2.840.113619.2.55.3.987654321",
    "image_count": 4,
    "compression_force": 120,
    "birads": "2"
  },
  "radiologist_report": {
    "provider_id": 8,
    "findings": "Scattered fibroglandular densities. No suspicious masses or calcifications.",
    "impression": "BIRADS 2 - Benign finding",
    "critical_finding": false,
    "signed_date": ISODate("2026-01-20T11:00:00Z")
  }
}
```

Embedding Rationale: DICOM metadata and radiologist report details are embedded because they are always accessed with the imaging study and vary significantly by modality. This avoids forcing CT (slice_thickness), MRI (sequence), X-ray (views), and mammography (birads) into one rigid SQL table with many NULL columns.

Expected Volume: ~120 documents seeded; production would see 2-10 imaging studies per patient per year.

Collection 3: care_plans

Purpose: Store care-management plans with nested goals and interventions for patients with chronic conditions.

Document Structure: - _id: ObjectId - patient_id: Integer, references PostgreSQL patient.patient_id - provider_id: Integer, references PostgreSQL provider.provider_id - primary_diagnosis: Embedded object with name, icd10, and category - status: String enum - active, completed, paused, cancelled - goals: Array of embedded goal objects with goal_id, description, target_date, status, progress_percent - interventions: Array of embedded intervention objects with intervention_id, description, owner_role, frequency - review_frequency_days: Integer review interval - last_reviewed: Most recent review date

Sample Document:

```
{
  "_id": ObjectId("..."),
  "patient_id": 3,
  "provider_id": 4,
  "primary_diagnosis": {
    "name": "Type 2 Diabetes",
    "icd10": "E11.9",
    "category": "Metabolic"
  },
  "status": "active",
  "goals": [
    {
      "goal_id": 1,
      "description": "Reduce HbA1c to below 7%",
      "target_date": ISODate("2026-06-01"),
      "status": "in_progress",
      "progress_percent": 65
    },
    {
      "goal_id": 2,
      "description": "Walk 30 minutes daily",
      "target_date": ISODate("2026-04-01"),
      "status": "completed",
      "progress_percent": 100
    }
  ],
  "interventions": [
    {
      "intervention_id": 1,
      "description": "Monthly HbA1c monitoring",
      "owner_role": "Nurse",
      "frequency": "monthly"
    }
  ]
}
```

```

    },
    {
      "intervention_id": 2,
      "description": "Diabetes education session",
      "owner_role": "Dietitian",
      "frequency": "quarterly"
    }
  ],
  "review_frequency_days": 30,
  "last_reviewed": ISODate("2026-03-01")
}

```

Embedding Rationale: Goals and interventions are embedded because each care plan has a bounded number of them (typically 3-8 goals) and care managers normally read or update them as part of the parent care plan. The primary diagnosis is embedded because it provides quick clinical context and includes the ICD-10 reference for filtering.

Expected Volume: ~80 documents seeded; production would see 1-5 active care plans per chronically ill patient.

Collection 4: patient_surveys

Purpose: Store patient-reported outcome surveys with variable question sets and scoring rules.

Document Structure: - `_id`: ObjectId - `patient_id`: Integer, references PostgreSQL `patient.patient_id` - `survey_type`: String enum - PHQ-9, GAD-7, Pain, Satisfaction - `completed_at`: Survey completion date - `responses`: Array of embedded response objects with `question_code`, `prompt`, `score`, `answer_text` - `score`: Embedded object with `total`, `max_possible`, and `severity` - `source`: Survey source - portal, tablet, phone, or paper - `notes`: Optional follow-up note

Sample Document (PHQ-9):

```

{
  "_id": ObjectId("..."),
  "patient_id": 7,
  "survey_type": "PHQ-9",
  "completed_at": ISODate("2026-03-10T09:15:00Z"),
  "responses": [
    {
      "question_code": "PHQ1",
      "prompt": "Little interest or pleasure in doing things",
      "score": 2,
      "answer_text": "More than half the days"
    },
    {
      "question_code": "PHQ2",
      "prompt": "Feeling down, depressed, or hopeless",
      "score": 1,
      "answer_text": "Several days"
    }
  ],
  "score": {
    "total": 12,
    "max_possible": 27,
    "severity": "moderate"
  }
}

```

```

    },
    "source": "tablet",
    "notes": "Patient reports improved sleep since last visit"
  }
}

```

Sample Document (Pain Assessment):

```

{
  "_id": ObjectId("..."),
  "patient_id": 15,
  "survey_type": "Pain",
  "completed_at": ISODate("2026-03-12T14:00:00Z"),
  "responses": [
    {
      "question_code": "PAIN1",
      "prompt": "Current pain level (0-10)",
      "score": 6,
      "answer_text": null
    },
    {
      "question_code": "PAIN2",
      "prompt": "Pain location",
      "score": null,
      "answer_text": "Lower back"
    },
    {
      "question_code": "PAIN3",
      "prompt": "Pain interferes with daily activities",
      "score": 7,
      "answer_text": null
    }
  ],
  "score": {
    "total": 6,
    "max_possible": 10,
    "severity": "moderate"
  },
  "source": "portal",
  "notes": null
}

```

Embedding Rationale: Responses are embedded because they are bounded by survey type (PHQ-9 has exactly 9 questions) and are always interpreted together with the parent survey score. MongoDB is appropriate because PHQ-9, GAD-7, pain assessments, and satisfaction surveys have entirely different structures and scoring algorithms.

Expected Volume: ~160 documents seeded; production would see 4-12 surveys per patient per year.

Part 3: MongoDB Index Strategy

Collection: `clinical_notes`

Index	Type	Fields	Purpose
Patient Timeline	Compound	{ patient_id: 1, encounter_date: -1 }	Efficiently retrieve notes for a patient in reverse chronological order
Provider Documentation	Compound	{ provider_id: 1, note_type: 1, encounter_date: -1 }	Provider workload analysis and documentation volume reporting
ICD-10 Filtering	Single	{ icd10_codes: 1 }	Filter notes by diagnosis code for clinical research
Narrative Search	Text	chief_complaint, assessment, plan, findings, recommendations, discharge_diagnosis	Full-text search across clinical narratives
Unsigned Notes	Partial	{ provider_id: 1, encounter_date: -1 } where signed: false	Provider inbox showing notes awaiting signature

Collection: medical_images_metadata

Index	Type	Fields	Purpose
Patient Timeline	Compound	{ patient_id: 1, study_date: -1 }	Retrieve imaging history for a patient
Imaging Volume	Compound	{ modality: 1, body_part: 1, study_date: -1 }	Volume analysis by modality and body region
Study UID	Unique	{ "dicom_metadata.study_uid": 1 }	Prevent duplicate imaging studies
Critical Findings	Partial	{ "radiologist_report.provider_id": 1, study_date: -1 } where radiologist_report.critical_finding: true	Alert queue for critical findings requiring radiologist follow-up
Report Search	Text	study_description, radiologist_report.findings, radiologist_report.impression	Search radiologist interpretations

Collection: care_plans

Index	Type	Fields	Purpose
Patient Care Plans	Compound	{ patient_id: 1, status: 1, last_reviewed: -1 }	Active care plan lookup by patient
Provider Workload	Compound	{ provider_id: 1, status: 1 }	Care coordinator caseload
Diagnosis Filter	Single	{ "primary_diagnosis.icd10": 1 }	Filter care plans by diagnosis

Index	Type	Fields	Purpose
Goal Status	Multikey	{ "goals.status": 1 }	Find care plans with incomplete goals
Care Plan Search	Text	primary_diagnosis.name, goals.description, interventions.description	Search care plan content

Collection: patient_surveys

Index	Type	Fields	Purpose
Patient Survey History	Compound	{ patient_id: 1, survey_type: 1, completed_at: -1 }	Patient's survey history by type
Risk Stratification	Compound	{ survey_type: 1, "score.severity": 1 }	Identify high-risk patients by survey type
Response Analysis	Multikey	{ "responses.question_code": 1 }	Question-level analysis across patients
Notes Search	Text	notes	Search clinical follow-up notes

Part 4: Neo4j Node and Relationship Catalog

Node Types

Node: Medication Properties: - med_id: Integer - Unique identifier matching PostgreSQL medication.med_id - name: String - Brand name (e.g., "Lipitor") - generic_name: String - Generic name (e.g., "Atorvastatin") - form: String - Dosage form (Tablet, Capsule, Injection, Inhaler, etc.) - is_controlled: Boolean - Whether it's a controlled substance - schedule: String (optional) - DEA schedule for controlled substances

Sample Nodes: Lipitor, Lisinopril, Metformin, Oxycodone, Warfarin, Sertraline, Omeprazole

Approximate Count: 35 medications

Constraints: med_id unique, name indexed

Node: DrugClass Properties: - name: String - Drug class name (e.g., "ACE Inhibitor") - description: String - Brief description of the drug class

Sample Nodes: Statin, ACE Inhibitor, SSRI, Opioid Analgesic, Anticoagulant, Benzodiazepine, PPI

Approximate Count: 26 drug classes

Constraints: name unique

Node: Disease Properties: - name: String - Disease name (e.g., "Hypertension") - icd10: String - ICD-10 diagnosis code (e.g., "I10") - category: String - Disease category (Cardiovascular, Metabolic, Mental Health, etc.) - chronic: Boolean - Whether it's a chronic condition

Sample Nodes: Hypertension (I10), Type 2 Diabetes (E11.9), Heart Failure (I50.9), Major Depression (F32.9), GERD (K21.0), Atrial Fibrillation (I48.91)

Approximate Count: 12 diseases

Constraints: icd10 unique, name indexed

Node: Symptom Properties: - name: String - Symptom name (e.g., “Chest Pain”) - body_system: String - Affected body system (Cardiovascular, Neurological, etc.) - severity_range: String - Typical severity range (e.g., “mild-severe”)

Sample Nodes: Chest Pain, Shortness of Breath, Fatigue, Headache, Dizziness, Palpitations, Edema

Approximate Count: 12 symptoms

Constraints: name unique, body_system indexed

Node: Allergen Properties: - name: String - Allergen name (e.g., “Penicillin”) - category: String - Allergen category (Drug, Food, Environmental, Insect)

Sample Nodes: Penicillin, Peanuts, Latex, Shellfish, Sulfa, Codeine, Ibuprofen

Approximate Count: 12 allergens

Constraints: name unique

Node: Patient Properties: - patient_id: Integer - Matches PostgreSQL patient.patient_id - mrn: String - Medical record number - first_name: String - last_name: String

Sample Nodes: Patient 1 (James Smith), Patient 2 (Patricia Brown), Patient 3 (Robert Miller)

Approximate Count: 10 patients (subset for graph queries)

Constraints: patient_id unique, mrn indexed

Node: Provider Properties: - provider_id: Integer - Matches PostgreSQL provider.provider_id - npi: String - National Provider Identifier - first_name: String - last_name: String - specialty: String - Medical specialty

Sample Nodes: Dr. Kelly Mbenga (Cardiologist), Dr. Michael Brown (Family Physician), Dr. Thomas Rodriguez (Oncologist)

Approximate Count: 9 providers

Constraints: provider_id unique, npi indexed, specialty indexed

Node: Department Properties: - dept_id: Integer - Matches PostgreSQL department.dept_id - name: String - Department name - hospital_id: Integer - Reference to hospital

Sample Nodes: Cardiology, Emergency Medicine, Family Medicine, Oncology

Approximate Count: 9 departments

Constraints: dept_id unique

Node: Hospital **Properties:** - **hospital_id:** Integer - Matches PostgreSQL hospital.hospital_id - **name:** String - Hospital name - **facility_type:** String - Hospital or Clinic - **city:** String - **state:** String

Sample Nodes: New York General Hospital, Los Angeles Medical Center, Chicago Central Hospital

Approximate Count: 5 hospitals

Constraints: hospital_id unique, name indexed

Relationship Types

Relationship: INTERACTS_WITH **From:** Medication **To:** Medication **Direction:** Bidirectional
(create both directions for each interaction)

Properties: - **severity:** String - “contraindicated”, “major”, “moderate”, “minor” - **description:** String
- Clinical description of the interaction

Examples:

(Oxycodone)-[:INTERACTS_WITH {severity: "contraindicated", description: "Risk of respiratory depression"}]->(Naproxen)

(Warfarin)-[:INTERACTS_WITH {severity: "major", description: "Increased bleeding risk"}]->(Naproxen)

(Sertraline)-[:INTERACTS_WITH {severity: "major", description: "Serotonin syndrome risk"}]->(Tramadol)

(Amlodipine)-[:INTERACTS_WITH {severity: "moderate", description: "Increased statin levels"}]->(Lipitor)

Approximate Count: 15+ interaction relationships

Clinical Use: Drug interaction checking at prescription time

Relationship: BELONGS_TO **From:** Medication **To:** DrugClass **Direction:** Medication -> DrugClass

Properties: None

Examples:

(Lipitor)-[:BELONGS_TO]->(Statin)

(Lisinopril)-[:BELONGS_TO]->(ACE Inhibitor)

(Sertraline)-[:BELONGS_TO]->(SSRI)

(Oxycodone)-[:BELONGS_TO]->(Opioid Analgesic)

Approximate Count: 35 relationships (one per medication)

Clinical Use: Drug class-level interaction analysis

Relationship: TREATED_BY **From:** Disease **To:** Medication **Direction:** Disease -> Medication

Properties: - **evidence_level:** String - “A” (strong), “B” (moderate), “C” (weak) - **first_line:** Boolean
- Whether it’s a first-line treatment

Examples:

(Hypertension)-[:TREATED_BY {evidence_level: "A", first_line: true}]->(Lisinopril)
(Type 2 Diabetes)-[:TREATED_BY {evidence_level: "A", first_line: true}]->(Metformin)
(Major Depression)-[:TREATED_BY {evidence_level: "A", first_line: true}]->(Sertraline)
(Chronic Pain)-[:TREATED_BY {evidence_level: "B", first_line: false}]->(Tramadol)

Approximate Count: 20+ treatment relationships

Clinical Use: Treatment pathway recommendations

Relationship: PRESENTS_WITH **From:** Disease **To:** Symptom **Direction:** Disease -> Symptom

Properties: - frequency: String - "very common", "common", "uncommon", "rare"

Examples:

(Hypertension)-[:PRESENTS_WITH {frequency: "common"}]->(Headache)
(Heart Failure)-[:PRESENTS_WITH {frequency: "very common"}]->(Shortness of Breath)
(Type 2 Diabetes)-[:PRESENTS_WITH {frequency: "very common"}]->(Fatigue)
(Atrial Fibrillation)-[:PRESENTS_WITH {frequency: "very common"}]->(Palpitations)

Approximate Count: 19 symptom relationships

Clinical Use: Differential diagnosis support

Relationship: CONTRAINDICATED_IN **From:** Medication **To:** Disease **Direction:** Medication
-> Disease

Properties: - reason: String - Clinical reason for contraindication

Examples:

(Metformin)-[:CONTRAINDICATED_IN {reason: "Risk of lactic acidosis"}]->(Kidney Disease)
(Naproxen)-[:CONTRAINDICATED_IN {reason: "Fluid retention and worsening heart failure"}]->(Heart Failure)
(Warfarin)-[:CONTRAINDICATED_IN {reason: "Impaired clotting factor synthesis"}]->(Liver Disease)

Approximate Count: 7+ contraindication relationships

Clinical Use: Disease-based medication safety checking

Relationship: CONTRAINDICATED_FOR **From:** Medication **To:** Allergen **Direction:** Medication
-> Allergen

Properties: - reason: String - Reason for allergy contraindication

Examples:

(Amoxicillin)-[:CONTRAINDICATED_FOR {reason: "Penicillin-class antibiotic"}]->(Penicillin)
(Naproxen)-[:CONTRAINDICATED_FOR {reason: "NSAID cross-reactivity"}]->(Ibuprofen)
(Codeine Syrup)-[:CONTRAINDICATED_FOR {reason: "Contains codeine"}]->(Codeine)

Approximate Count: 5+ allergen contraindication relationships

Clinical Use: Allergy-based prescription safety

Relationship: ALLERGIC_TO **From:** Patient **To:** Allergen **Direction:** Patient -> Allergen

Properties: - severity: String - "Low", "Moderate", "High", "Critical" - reaction: String - Description of allergic reaction

Examples:

(Patient 1)-[:ALLERGIC_TO {severity: "Low", reaction: "Rash"}]->(Penicillin)
(Patient 6)-[:ALLERGIC_TO {severity: "Moderate", reaction: "Anaphylaxis"}]->(Ibuprofen)
(Patient 4)-[:ALLERGIC_TO {severity: "Critical", reaction: "Sneezing"}]->(Shellfish)

Approximate Count: 10+ patient allergy relationships

Clinical Use: Patient-specific medication safety checking

Relationship: REFERS_TO **From:** Provider **To:** Provider **Direction:** Provider -> Provider

Properties: - referral_count: Integer - Number of referrals - common_reasons: List - Common referral reasons

Examples:

(Family Physician)-[:REFERS_TO {referral_count: 45, common_reasons: ["chest pain", "hypertension"]}]->(Emergency Physician)
(Emergency Physician)-[:REFERS_TO {referral_count: 67, common_reasons: ["chest pain", "heart attack"]}]->(Internist)
(Internist)-[:REFERS_TO {referral_count: 18, common_reasons: ["abnormal labs", "mass found"]}]->(Oncologist)

Approximate Count: 5+ referral relationships

Clinical Use: Provider network analysis

Relationship: WORKS_IN **From:** Provider **To:** Department **Direction:** Provider -> Department

Properties: None

Examples:

(Dr. Kelly Mbenga)-[:WORKS_IN]->(Cardiology)
(Dr. Michael Brown)-[:WORKS_IN]->(Family Medicine)
(Dr. Thomas Rodriguez)-[:WORKS_IN]->(Oncology)

Approximate Count: 9 relationships (one per provider)

Clinical Use: Organizational hierarchy

Relationship: PART_OF **From:** Department **To:** Hospital **Direction:** Department -> Hospital

Properties: None

Examples:

(Cardiology)-[:PART_OF]->(New York General Hospital)
(Family Medicine)-[:PART_OF]->(Los Angeles Medical Center)
(Oncology)-[:PART_OF]->(Miami Wellness Hospital)

Approximate Count: 9 relationships (one per department)

Clinical Use: Organizational hierarchy queries

Relationship: TREATED_BY (Patient -> Provider) **From:** Patient **To:** Provider **Direction:** Patient -> Provider

Properties: None

Examples:

(Patient 1)-[:TREATED_BY]->(Dr. Kelly Mbenga)

(Patient 2)-[:TREATED_BY]->(Dr. Michael Brown)

Approximate Count: 5+ patient-provider relationships

Clinical Use: Care team identification

Graph Statistics Summary

Element	Count
Node Types	9
Medication Nodes	35
Disease Nodes	12
Symptom Nodes	12
DrugClass Nodes	26
Allergen Nodes	12
Patient Nodes	10
Provider Nodes	9
Department Nodes	9
Hospital Nodes	5
Relationship Types	10
INTERACTS_WITH	15+
TREATED_BY (Disease->Med)	20+
PRESENTS_WITH	19
CONTRAINDICATED_IN	7+
CONTRAINDICATED_FOR	5+
BELONGS_TO	35
ALLERGIC_TO	10+

Cross-Database Reference Strategy

PostgreSQL as Source of Truth

- Patient demographics, provider credentials, and medication catalogs remain in PostgreSQL with ACID guarantees
- MongoDB documents reference PostgreSQL IDs (`patient_id`, `provider_id`) but do not duplicate full records
- Neo4j nodes mirror key identifiers (`patient_id`, `provider_id`, `med_id`) for graph queries

Data Synchronization Approach

- PostgreSQL IDs are immutable and serve as cross-database join keys
- MongoDB and Neo4j are populated during initial data load
- Production would use event-driven synchronization for new patients/providers

Query Routing

- Transactional operations (prescriptions, appointments) → PostgreSQL
 - Document retrieval and text search (clinical notes) → MongoDB
 - Relationship queries (drug interactions, pathways) → Neo4j
 - Unified operations compose results from all three databases
-